Mortgage pricing exceptions, with a traceable decision

An internal workflow for requesting a mortgage rate discount and reviewing the exact terms independently.

Request

A relationship manager proposes a discount in basis points, provides a reason, and assigns a reviewer.

Review

Only the assigned independent reviewer can approve or decline the current revision.

Retrieve

The mortgage process retrieves a discount only when the current terms have an effective approval.

Workflow controls

No self-review • One request per application • Retry safety • Preserved history

Create → assign → decide → use the approved discount

Three relationship managers and five reviewers use predefined development identities.

01

Manager creates

James submits terms and selects a reviewer.

02

Reviewer decides

David, Emily, or Robert records a decision and reason.

03

Process retrieves

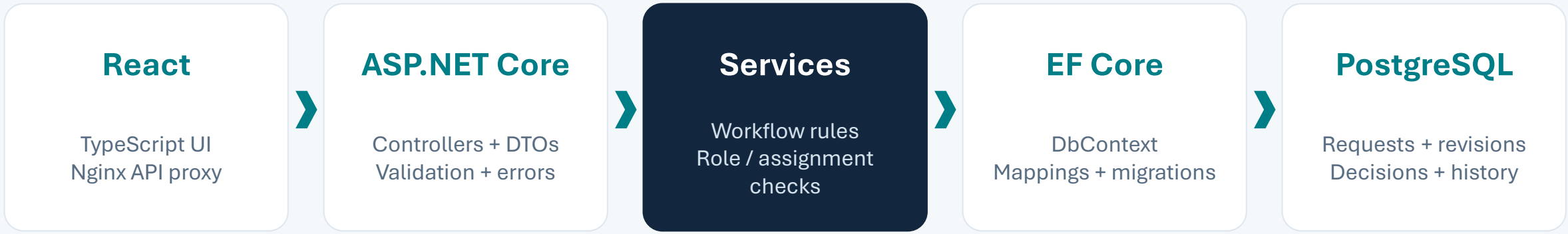
Approved current terms are available for processing.

Changed terms require a fresh decision

A revision returns the request to Pending; the previous approval remains in history but stops being effective.

A layered application with PostgreSQL persistence

React handles interaction; ASP.NET Core enforces the workflow; PostgreSQL stores durable records.



Service boundaries

PricingExceptionService • UserService • ApprovedDiscountService

Docker Compose

Three containers; a named volume preserves PostgreSQL data.

Protect the terms, the history, and the retry

Business rules live in services, with explicit HTTP contracts and testable dependencies.

Revisions + stale-write checks

New terms create a new revision. Expected version and revision checks reject outdated edits or decisions.

Preserved request history

Submitted terms and decisions remain immutable. Actor IDs, name snapshots, and UTC timestamps retain attribution.

Idempotent operations

Same key + same input replays the original result. Reusing the key with changed input returns 409 Conflict.

Dependency injection

Scoped services and DbContext keep responsibilities clear. Interfaces and TimeProvider support isolated testing.

Verified workflows, with transparent AI assistance

Local validation results from the implemented assignment; no claim of production readiness.

69

.NET tests

24 unit + 45 HTTP integration

13

Browser tests

User switching + workflow + mobile

Docker / PostgreSQL checks passed

Three-request browser demo, API smoke tests, and reseeding without duplicate users or record loss.

How Codex supported development

Drafted and refined code, tests, Docker setup, documentation, and debugging.

Developer-directed steps; generated changes require review and test evidence.

LIVE DEMO James creates 3 requests → David / Emily / Robert decide → inspect history